

Functional Programming

Algebraic datatypes Part 2

Jesper Bengtson

Last weeks

We covered algebraic datatypes

- In depth
 - ▶ Inductively defined datatypes
 - ▶ Data representation
 - ▶ Option, Result, and bind
- Ever so briefly
 - ▶ Expression trees
 - ▶ Collections (Sets and maps)
 - ▶ Not at all
 - ▶ Variable state for your interpreter

Questions?

Inductively defined types

- ... or algebraic datatypes
- ... or disjoint unions
- ... or discriminate union

Allow us to concisely say how members of types are created

The option type

Options are used to encode partial functions

```
type 'a option =  
| None          (* No result *)  
| Some of 'a    (* result *)
```

You can think of options as terms that can be set to null (but nice, and type-safe, and does not cause as many bugs)

Head and tails of a list

The functions `List.head` and `List.tail` will throw an exception if you try to get the head of an empty list (`List.head []` or `List.tail []`)

```
let safeHead =  
  function  
  | []      -> None  
  | x :: _  -> Some x
```

```
let safeTail =  
  function  
  | []      -> None  
  | _ :: xs -> Some xs
```

Bind

We have seen these before

```
Option.get : 'a option -> 'a
```

```
Option.map : ('a -> 'b) -> 'a option -> 'b option
```

```
Option.defaultValue : 'a -> 'a option -> 'a
```

But not one of the most useful ones

```
Option.bind :
```

```
  ('a -> 'b option) -> 'a option -> 'b option
```

Bind

```
Option.bind :  
  ('a -> 'b option) -> 'a option -> 'b option  
  
let bind binder opt =  
  match opt with  
  | None      -> None  
  | Some x    -> binder x
```

The bind function unfolds the option. The intuition is that if a result ever becomes None, it will stay there.

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let safeHead =
  function
  | []          -> None
  | x :: _     -> Some x
```

```
let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x
```

```
let safeTail =
  function
  | []          -> None
  | _ :: xs     -> Some xs
```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x
```

```
let safeHead =
  function
  | []      -> None
  | x :: _  -> Some x

let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some 4
```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x
```

```
let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
[1;2;3;4;5]
```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x
```

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some [2;3;4;5]
```

```
let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x
```

```
let safeHead =
  function
  | []      -> None
  | x :: _  -> Some x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some [3;4;5]
```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x

let safeHead =
  function
  | []        -> None
  | x :: _    -> Some x

let safeTail =
  function
  | []        -> None
  | _ :: xs   -> Some xs

[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some [4;5]

```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x

let safeHead =
  function
  | []      -> None
  | x :: _  -> Some x

let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs

[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some 4

```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x
```

```
let safeHead =
  function
  | []      -> None
  | x :: _  -> Some x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
None
```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x
```

```
let safeHead =
  function
  | []      -> None
  | x :: _  -> Some x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
[1;2]
```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x

let safeHead =
  function
  | []        -> None
  | x :: _    -> Some x

let safeTail =
  function
  | []        -> None
  | _ :: xs   -> Some xs

[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some [2]

```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x

let safeHead =
  function
  | []        -> None
  | x :: _    -> Some x

let safeTail =
  function
  | []        -> None
  | _ :: xs   -> Some xs

[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some []

```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x

let safeHead =
  function
  | []        -> None
  | x :: _    -> Some x

let safeTail =
  function
  | []        -> None
  | _ :: xs   -> Some xs

[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
None

```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder opt =
  match opt with
  | None      -> None
  | Some x    -> binder x

let safeHead =
  function
  | []        -> None
  | x :: _    -> Some x

let safeTail =
  function
  | []        -> None
  | _ :: xs   -> Some xs

[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
None

```

Bind

```
Option.bind :          let bind binder opt =
  ('a -> 'b option) ->    match opt with
  'a option ->           | None    -> None
```

The bind function unfolds the option. The intuition is that if a result ever becomes None, it will stay there.

```

| []      -> None
| x :: _ -> Some x
```

```

| []      -> None
| _ :: xs -> Some xs
```

```
[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
None
```

Bind

```
Option.bind :           let bind binder opt =
  ('a -> 'b option) ->   match opt with
  'a option ->           | None    -> None
```

The bind function unfolds the option. The intuition is that if a result ever becomes None, it will stay there.

The function safeHead is never even called!

```
[ : ]
Option.bind safeTail |> Option.bind safeHead =
None
```

Bind is universal

When we come to railroad oriented programming (monads) you will use bind a lot so get used to it. It's an amazing abstraction and we are just scratching the surface here.

Bind is universal

When we come to railroad oriented programming (monads) you will use bind a lot so get used to it. It's an amazing abstraction and we are just scratching the surface here.

```
let (>>=) x f = x |> Option.bind f
```

```
[1;2;3;4;5] |> safeTail >>= safeTail >>=
safeTail >>= safeHead = Some 4
```

instead of

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some 4
```

Bind is universal

We are getting ahead of ourselves. We will talk much more about the uses of bind and the `>>=` operator in a few weeks, but you can do amazing things

```
let (>>=) x f = x |> Option.bind f
```

```
[1;2;3;4;5] |> safeTail >>= safeTail >>=
safeTail >>= safeHead = Some 4
```

instead of

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some 4
```

Obtaining multiple arguments

It is common to nest binds to obtain multiple results to use later. This function takes the head of two lists and returns them as a pair

```
let safeHeads xs ys =  
  Option.bind (fun x ->  
    Option.bind (fun y -> Some (x, y))  
                (safeHead ys))  
              (safeHead xs)
```

```
safeHeads [1;2] [3;4] = Some (1, 3)
```

Obtaining multiple arguments

It is common to nest binds to obtain multiple results to use later. This function takes the head of two lists and returns them as a pair

```
let safeHeads xs ys =  
  Option.bind (fun x ->  
    Option.bind (fun y -> Some (x, y))  
                (safeHead ys))  
              (safeHead xs)
```

```
safeHeads [1;2] [] = None
```

Obtaining multiple arguments

Can be written cleaner with `>>=`

```
let safeHeads xs ys =  
  Option.bind (fun x ->  
    Option.bind (fun y -> Some (x, y))  
                (safeHead ys))  
              (safeHead xs)
```

```
let safeHeads xs ys =  
  safeHead xs >>= (fun x ->  
    safeHead ys >>= (fun y ->  
      Some (x, y)))
```

Questions?

Collections

F# has support for all collections from the .NET framework

- Lists
- Sets
- Maps
- Hash tables
- ...

Sets

- Represents mathematical sets
- Intersection, union, ...
- Not mutable
- Unordered
- Can only store values of comparison type
- Created inline by `set [a1; a2; ...; an]` where all duplicates are removed

Sets (some functions)

`Set.empty` : `Set<'a>`

`Set.empty = set []`

Sets (some functions)

```
Set.empty      : Set<'a>  
Set.singleton : 'a -> Set<'a>
```

```
Set.singleton 42 = set [42]
```

Sets (some functions)

```
Set.empty      : Set<'a>  
Set.singleton  : 'a -> Set<'a>  
Set.union      : Set<'a> -> Set<'a> -> Set<'a>
```

```
Set.union (Set.singleton 42)  
  (Set.singleton 123) = set [42; 123]
```

Sets (some functions)

```
Set.empty      : Set<'a>  
Set.singleton  : 'a -> Set<'a>  
Set.union      : Set<'a> -> Set<'a> -> Set<'a>  
Set.intersect  : Set<'a> -> Set<'a> -> Set<'a>
```

```
Set.intersect (set [1;2;3]) (set [3;4;5]) = set [3]
```

Sets (some functions)

`Set.empty` : `Set<'a>`

`Set.singleton` : `'a -> Set<'a>`

`Set.union` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.intersect` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.map` : `('a -> 'b) -> Set<'a> -> Set<'b>`

`Set.map (fun x -> x + 2) (set [3;4;5]) = set [5;6;7]`

Sets (some functions)

`Set.empty` : `Set<'a>`

`Set.singleton` : `'a -> Set<'a>`

`Set.union` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.intersect` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.map` : `('a -> 'b) -> Set<'a> -> Set<'b>`

`Set.map (fun x -> x + 2) (set [3;4;5]) = set [5;6;7]`

`Set.map ((+) 2) (set [3;4;5]) = set [5;6;7]`

Sets (some functions)

`Set.empty` : `Set<'a>`

`Set.singleton` : `'a -> Set<'a>`

`Set.union` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.intersect` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.map` : `('a -> 'b) -> Set<'a> -> Set<'b>`

`Set.map (fun x -> x + 2) (set [3;4;5]) = set [5;6;7]`

`Set.map ((+) 2) (set [3;4;5]) = set [5;6;7]`

`Set.map (fun x -> x % 2 = 0) (set [3;4;5]) =
set [false; true]`

Sets (some functions)

`Set.empty` : `Set<'a>`

`Set.singleton` : `'a -> Set<'a>`

`Set.union` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.intersect` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.map` : `('a -> 'b) -> Set<'a> -> Set<'b>`

`Set.filter` : `('a -> bool) -> Set<'a> -> Set<'a>`

`Set.filter (fun x->x%2=0) (set[3;4;5]) = set[4]`

Sets (some functions)

`Set.empty` : `Set<'a>`

`Set.singleton` : `'a -> Set<'a>`

`Set.union` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.intersect` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.map` : `('a -> 'b) -> Set<'a> -> Set<'b>`

`Set.filter` : `('a -> bool) -> Set<'a> -> Set<'a>`

`Set.filter (fun x->x%2=0) (set[3;4;5]) = set[4]`

`Set.filter (fun x->x%2=1) (set[3;4;5]) = set[3;5]`

Sets (some functions)

```
Set.empty      : Set<'a>
Set.singleton  : 'a -> Set<'a>
Set.union      : Set<'a> -> Set<'a> -> Set<'a>
Set.intersect  : Set<'a> -> Set<'a> -> Set<'a>

Set.map        : ('a -> 'b) -> Set<'a> -> Set<'b>
Set.filter     : ('a -> bool) -> Set<'a> -> Set<'a>
Set.fold       : ('b -> 'a -> 'b) -> 'b ->
                Set<'a> -> 'b

Set.fold (fun x y -> x + y) 0 (set[3;4;5]) = 12
```

Sets (some functions)

`Set.empty` : `Set<'a>`

`Set.singleton` : `'a -> Set<'a>`

`Set.union` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.intersect` : `Set<'a> -> Set<'a> -> Set<'a>`

`Set.map` : `('a -> 'b) -> Set<'a> -> Set<'b>`

`Set.filter` : `('a -> bool) -> Set<'a> -> Set<'a>`

`Set.fold` : `('b -> 'a -> 'b) -> 'b ->
Set<'a> -> 'b`

`Set.fold (fun x y -> x + y) 0 (set[3;4;5]) = 12`

`Set.fold (+) 0 (set[3;4;5]) = 12`

Sets (some functions)

```
Set.empty
```

Result: `set []`

Sets (some functions)

```
Set.empty |>  
Set.add "hello"
```

Result: set ["hello"]

Sets (some functions)

```
Set.empty |>  
Set.add "hello" |>  
Set.add "there"
```

Result: set ["hello"; "there"]

Sets (some functions)

```
Set.empty |>  
Set.add "hello" |>  
Set.add "there" |>  
Set.union (Set.ofList [ "there"; "you"; "are" ])
```

Result: set ["hello"; "there"; "you"; "are"]

Sets (some functions)

```
Set.empty |>  
Set.add "hello" |>  
Set.add "there" |>  
Set.union (Set.ofList ["there"; "you"; "are"]) |>  
Set.intersect (Set.ofList ["hello"; "there"; "you"])
```

Result: set ["hello"; "there"; "you"]

Sets (some functions)

```
Set.empty |>  
Set.add "hello" |>  
Set.add "there" |>  
Set.union (Set.ofList ["there"; "you"; "are"]) |>  
Set.intersect (Set.ofList ["hello"; "there"; "you"]) |>  
Set.map String.length
```

Result: set [5; 3]

Sets (some functions)

```
Set.empty |>  
Set.add "hello" |>  
Set.add "there" |>  
Set.union (Set.ofList ["there"; "you"; "are"]) |>  
Set.intersect (Set.ofList ["hello"; "there"; "you"]) |>  
Set.map String.length |>  
Set.filter (fun x -> x % 2 == 1)
```

Result: set [5; 3]

Sets (some functions)

```
Set.empty |>  
Set.add "hello" |>  
Set.add "there" |>  
Set.union (Set.ofList ["there"; "you"; "are"]) |>  
Set.intersect (Set.ofList ["hello"; "there"; "you"]) |>  
Set.map String.length |>  
Set.filter (fun x -> x % 2 = 1) |>  
Set.fold (+) 0
```

Result: 8

Maps

- Represents mathematical maps
- add, lookup, ...
- Not mutable
- Unordered
- Can only have keys of comparison type

Maps (some functions)

```
Map.empty      : Map<'a, 'b>
```

```
Map.empty = map []
```

Maps (some functions)

`Map.empty` : `Map<'a, 'b>`

`Map.add` : `'a -> 'b -> Map<'a, 'b> -> Map<'a, 'b>`

`Map.add "k" 4 Map.empty = map [ ("k", 4) ]`

Maps (some functions)

```
Map.empty      : Map<'a, 'b>  
Map.add        : 'a -> 'b -> Map<'a, 'b> -> Map<'a, 'b>  
Map.ofList     : ('a*'b) list -> Map<'a, 'b>
```

```
Map.ofList [ ("k", 4); ("l", 5) ] =  
  map [ ("k", 4); ("l", 5) ]
```

Maps (some functions)

```
Map.empty      : Map<'a, 'b>
Map.add        : 'a -> 'b -> Map<'a, 'b> -> Map<'a, 'b>
Map.ofList     : ('a*'b) list -> Map<'a, 'b>
Map.find       : 'a -> Map<'a, 'b> -> 'b
```

`Map.find "k" (Map.add "k" 4 Map.empty) = 4`

which is the same as

`Map.empty |> Map.add "k" 4 |> Map.find "k" = 4`

Maps (some functions)

```
Map.empty      : Map<'a, 'b>  
Map.add        : 'a -> 'b -> Map<'a, 'b> -> Map<'a, 'b>  
Map.ofList     : ('a*'b) list -> Map<'a, 'b>  
Map.find       : 'a -> Map<'a, 'b> -> 'b
```

```
Map.find "l" (Map.add "k" 4 Map.empty)
```

Both are identical and throw exceptions

```
Map.empty |> Map.add "k" 4 |> Map.find "l"
```

Maps (some functions)

```
Map.empty      : Map<'a, 'b>
Map.add        : 'a -> 'b -> Map<'a, 'b> -> Map<'a, 'b>
Map.ofList     : ('a*'b) list -> Map<'a, 'b>
Map.find       : 'a -> Map<'a, 'b> -> 'b
Map.tryFind    : 'a -> Map<'a, 'b> -> 'b option
```

```
Map.tryFind "k" (Map.add "k" 4 Map.empty) = Some 4
```

which is the same as

```
Map.empty |> Map.add "k" 4 |> Map.tryFind "k" =
  Some 4
```

Maps (some functions)

```
Map.empty      : Map<'a, 'b>
Map.add        : 'a -> 'b -> Map<'a, 'b> -> Map<'a, 'b>
Map.ofList     : ('a*'b) list -> Map<'a, 'b>
Map.find       : 'a -> Map<'a, 'b> -> 'b
Map.tryFind    : 'a -> Map<'a, 'b> -> 'b option
```

```
Map.map        : ('a->'b->'c) -> Map<'a, 'b> -> Map<'a, 'c>
```

```
Map.map (fun k v->k+string v) (Map.ofList [ ("k",4); ("l",5) ]) =
  map [ ("k", "k4"); ("l", "l5") ]
```

which is the same as

```
[ ("k",4); ("l",5) ] |>Map.ofList |>Map.map (fun k v->k+string v) =
  map [ ("k", "k4"); ("l", "l5") ]
```

Maps (some functions)

```
Map.empty      : Map<'a, 'b>
Map.add        : 'a -> 'b -> Map<'a, 'b> -> Map<'a, 'b>
Map.ofList     : ('a*'b) list -> Map<'a, 'b>
Map.find       : 'a -> Map<'a, 'b> -> 'b
Map.tryFind    : 'a -> Map<'a, 'b> -> 'b option

Map.map        : ('a->'b->'c) -> Map<'a, 'b> -> Map<'a, 'c>
Map.filter     : ('a->'b->bool) -> Map<'a, 'b> -> Map<'a, 'b>

Map.filter (fun _ v->v%2=0) (Map.ofList [("k",4);("l",5)]) =
  map [("k",4)]
```

Maps (some functions)

```

Map.empty      : Map<'a, 'b>
Map.add        : 'a -> 'b -> Map<'a, 'b> -> Map<'a, 'b>
Map.ofList     : ('a*'b) list -> Map<'a, 'b>
Map.find       : 'a -> Map<'a, 'b> -> 'b
Map.tryFind    : 'a -> Map<'a, 'b> -> 'b option

Map.map        : ('a->'b->'c) -> Map<'a, 'b> -> Map<'a, 'c>
Map.filter     : ('a->'b->bool) -> Map<'a, 'b> -> Map<'a, 'b>
Map.fold       : ('c->'a->'b->'c) -> 'c -> Map<'a, 'b> -> 'c

Map.fold (fun acc k v->acc+k+string v)
  ""
  (Map.ofList [("k",4);("l",5)]) = "k4l5"

```

Higher-order functions

For lists you can potentially get away without using higher-order functions by using recursion

For sets and maps you still can (translate them to lists, work on the lists and then translate them back) but this is a **really** bad idea.

Practice using higher-order functions :)

Program interpreters

Functional languages are great for
working directly on abstract syntax trees

A small imperative language

$$a ::= \mathbb{Z}$$

	v
	$a + a$
	$a \times a$
	a / a

$$b ::= \top$$

	$a = a$
	$a < a$
	$b \wedge b$
	$\neg b$

$$S ::= \epsilon$$

	declare v
	$v \leftarrow a$
	$S; S$
	if b then S else S
	while b do S

Arithmetic expressions

An arithmetic expression is either

$$a ::= \boxed{\mathbb{Z}} \\ \quad | v \\ \quad | a + a \\ \quad | a \times a \\ \quad | a / a$$

An integer

Arithmetic expressions

An arithmetic expression is either

$$a ::= \mathcal{Z} \mid \boxed{v} \mid a + a \mid a \times a \mid a / a$$

A variable name

Arithmetic expressions

An arithmetic expression is either

$$a ::= \begin{array}{l} \mathbb{Z} \\ v \\ \boxed{a + a} \\ a \times a \\ a / a \end{array}$$

The addition of two
arithmetic expressions

Arithmetic expressions

An arithmetic expression is either

$$\begin{array}{l} a ::= \mathbb{Z} \\ \quad | v \\ \quad | a + a \\ \quad | \boxed{a \times a} \\ \quad | a / a \end{array}$$

The multiplication of two arithmetic expressions

Arithmetic expressions

An arithmetic expression is either

$$a ::= \begin{array}{l} \mathbb{Z} \\ v \\ a + a \\ a \times a \\ \boxed{a / a} \end{array}$$

The division of two
arithmetic expressions

Arithmetic expressions

Subtraction can be encoded

$$a_1 - a_2 = a_1 + (a_2 \times -1)$$

$$a ::= \begin{array}{l} \mathbb{Z} \\ v \\ a + a \\ a \times a \\ a / a \end{array}$$

Arithmetic expressions

$$\begin{array}{l} a ::= \mathbb{Z} \\ \quad | v \\ \quad | a + a \\ \quad | a \times a \\ \quad | a / a \end{array}$$

Subtraction can be encoded

$$a_1 - a_2 = a_1 + (a_2 \times -1)$$

Modulo can be encoded

$$a \% b = a - (a \text{ div } b) * b$$

Arithmetic expressions

Subtraction can be encoded

$$a_1 - a_2 = a_1 + (a_2 \times -1)$$

Modulo can be encoded

$$a \% b = a - (a \text{ div } b) * b$$

$$a ::= \begin{array}{l} \mathbb{Z} \\ v \\ a + a \\ a \times a \\ a / a \end{array}$$

We encode to simplify your implementation.
Your functions only need to reason about a
core set of arithmetic expressions, not
encodings of other operators

A small imperative language

$$a ::= \mathbb{Z}$$

	v
	$a + a$
	$a \times a$
	a / a

$$b ::= \top$$

	$a = a$
	$a < a$
	$b \wedge b$
	$\neg b$

$$S ::= \epsilon$$

	declare v
	$v \leftarrow a$
	$S; S$
	if b then S else S
	while b do S

Boolean expressions

An boolean expression is either

$$b ::= \boxed{\top} \mid a = a \mid a < a \mid b \wedge b \mid \neg b$$

True

Boolean expressions

$b ::= \top$

$a = a$

$a < a$

$b \wedge b$

$\neg b$

An boolean expression is either

Equality of two arithmetic
expressions

Boolean expressions

An boolean expression is either

Strictly less than of two
arithmetic expressions

$$\begin{array}{l} b ::= \top \\ \quad | a = a \\ \quad | \boxed{a < a} \\ \quad | b \wedge b \\ \quad | \neg b \end{array}$$

Boolean expressions

$$\begin{array}{l} b ::= \top \\ \quad | a = a \\ \quad | a < a \\ \quad | \boxed{b \wedge b} \\ \quad | \neg b \end{array}$$

An boolean expression is either

Conjunction of two boolean expressions

Boolean expressions

$$\begin{array}{l} b ::= \top \\ \quad | a = a \\ \quad | a < a \\ \quad | b \wedge b \\ \quad | \boxed{\neg b} \end{array}$$

An boolean expression is either

Negation (not) of one
boolean expression

Boolean expressions

$$b ::= \top$$

	$a = a$
	$a < a$
	$b \wedge b$
	$\neg b$

False can be encoded

$$\perp = \neg \top$$

Boolean expressions

$$b ::= \top$$

	$a = a$
	$a < a$
	$b \wedge b$
	$\neg b$

Inequality can be encoded

$$\perp = \neg \top$$
$$a_1 \neq a_2 = \neg(a_1 = a_2)$$

Boolean expressions

$$\begin{array}{l}
 b ::= \top \\
 \quad | a = a \\
 \quad | a < a \\
 \quad | b \wedge b \\
 \quad | \neg b
 \end{array}$$

Disjunction can be encoded

$$\begin{array}{lcl}
 \perp & = & \neg \top \\
 a_1 \neq a_2 & = & \neg(a_1 = a_2) \\
 b_1 \vee b_2 & = & \neg(\neg b_1 \wedge \neg b_2)
 \end{array}$$

Boolean expressions

Less than or equal to can be encoded

$$\begin{array}{l}
 b ::= \top \\
 \quad | a = a \\
 \quad | a < a \\
 \quad | b \wedge b \\
 \quad | \neg b
 \end{array}$$

$$\begin{array}{lcl}
 \perp & = & \neg \top \\
 a_1 \neq a_2 & = & \neg(a_1 = a_2) \\
 b_1 \vee b_2 & = & \neg(\neg b_1 \wedge \neg b_2) \\
 a_1 \leq a_2 & = & a_1 < a_2 \vee a_1 = a_2
 \end{array}$$

Boolean expressions

Greater than or equal to can be encoded

$$\begin{array}{l}
 b ::= \top \\
 \quad | a = a \\
 \quad | a < a \\
 \quad | b \wedge b \\
 \quad | \neg b
 \end{array}$$

$$\begin{array}{lcl}
 \perp & = & \neg \top \\
 a_1 \neq a_2 & = & \neg(a_1 = a_2) \\
 b_1 \vee b_2 & = & \neg(\neg b_1 \wedge \neg b_2) \\
 a_1 \leq a_2 & = & a_1 < a_2 \vee a_1 = a_2 \\
 a_1 \geq a_2 & = & \neg(a_1 < a_2)
 \end{array}$$

Boolean expressions

Strictly greater than can be encoded

$$\begin{array}{l}
 b ::= \top \\
 \quad | a = a \\
 \quad | a < a \\
 \quad | b \wedge b \\
 \quad | \neg b
 \end{array}$$

$$\begin{array}{lcl}
 \perp & = & \neg \top \\
 a_1 \neq a_2 & = & \neg(a_1 = a_2) \\
 b_1 \vee b_2 & = & \neg(\neg b_1 \wedge \neg b_2) \\
 a_1 \leq a_2 & = & a_1 < a_2 \vee a_1 = a_2 \\
 a_1 \geq a_2 & = & \neg(a_1 < a_2) \\
 a_1 > a_2 & = & a_1 \geq a_2 \wedge a_1 \neq a_2
 \end{array}$$

A small imperative language

$$a ::= \mathbb{Z}$$

	v
	$a + a$
	$a \times a$
	a / a

$$b ::= \top$$

	$a = a$
	$a < a$
	$b \wedge b$
	$\neg b$

$$S ::= \epsilon$$

	declare v
	$v \leftarrow a$
	$S; S$
	if b then S else S
	while b do S

Statements

$S ::= \boxed{\epsilon}$
| declare v
| $v \leftarrow a$
| $S; S$
| if b then S else S
| while b do S

A statement is either

The empty statement
(does nothing)

Statements

$S ::= \epsilon$

| **declare v**

| $v \leftarrow a$

| $S; S$

| if b then S else S

| while b do S

A statement is either

A variable declaration

Statements

$S ::= \epsilon$
| declare v
| $v \leftarrow a$
| $S; S$
| if b then S else S
| while b do S

A statement is either

A variable assignment

Statements

$S ::= \epsilon$
| declare v
| $v \leftarrow a$
| **$S; S$**
| if b then S else S
| while b do S

A statement is either

Sequencing of two
statements (one
statement after the
other)

Statements

$S ::= \epsilon$
| declare v
| $v \leftarrow a$
| $S; S$
| if b then S else S
| while b do S

A statement is either

A conditional statement
(if-then-else)

Statements

$S ::= \epsilon$
| declare v
| $v \leftarrow a$
| $S; S$
| if b then S else S
| **while b do S**

A statement is either

A while loop

Statements

$S ::= \epsilon$
| declare v
| $v \leftarrow a$
| $S; S$
| if b then S else S
| while b do S

If-then can be encoded

if b then S = if b then S else ϵ

Arithmetic expression implementation

$$a ::= \mathbb{Z}$$

$$| v$$

$$| a + a$$

$$| a \times a$$

$$| a / a$$

```

type aExp =
  | N of int
  | V of string
  | Add of (aExp * aExp)
  | Mul of (aExp * aExp)
  | Div of (aExp * aExp)

```

Arithmetic expression implementation

$$a ::= \mathbb{Z}$$

	v
	$a + a$
	$a \times a$
	a / a
	$a \bmod a$

```

type aExp =
  | N of int
  | V of string
  | Add of (aExp * aExp)
  | Mul of (aExp * aExp)
  | Div of (aExp * aExp)

```

	<code>let (.+.) a b = Add (a, b)</code>
$a_1 - a_2 =$	<code>let (.-.) a b = Add (a, (Mul b, N -1))</code>
$a_1 + (a_2 \times -1)$	<code>let (.*.) a b = Mul (a, b)</code>
	<code>let (./.) a b = Div (a, b)</code>
	<code>let (.%.) a b = a .-. ((a ./. b) .*. b)</code>

Boolean expression implementation

$$b ::= \top$$

	$a = a$
	$a < a$
	$b \wedge b$
	$\neg b$

```
type bExp =  
  TT  
  | Eq of (aExp * aExp)  
  | Lt of (aExp * aExp)  
  | Conj of (bExp * bExp)  
  | Not of bExp
```

Boolean expression implementation

$$b ::= \top$$

	$a = a$
	$a < a$
	$b \wedge b$
	$\neg b$

$$\perp = \neg \top$$

$$a_1 \neq a_2 = \neg(a_1 = a_2)$$

$$b_1 \vee b_2 = \neg(\neg b_1 \wedge \neg b_2)$$

```
type bExp =
  | TT
  | Eq of (aExp * aExp)
  | Lt of (aExp * aExp)
  | Conj of (bExp * bExp)
  | Not of bExp
```

```
let FF = Not TT
let (.<.>.) a b = ~~(a .=. b)
let (~~) b = Not b
let (.&&.) a b = Conj (a, b)
let (.||.) a b = ~~(~~a .&&. ~~b)
```

Boolean expression implementation

$$b ::= \top$$

$a = a$	
$a < a$	
$b \wedge b$	
$\neg b$	

$$\perp = \neg \top$$

$$a_1 \neq a_2 = \neg(a_1 = a_2)$$

$$b_1 \vee b_2 = \neg(\neg b_1 \wedge \neg b_2)$$

$$a_1 \leq a_2 = a_1 < a_2 \vee a_1 = a_2$$

$$a_1 \geq a_2 = \neg(a_1 < a_2)$$

$$a_1 > a_2 = a_1 \geq a_2 \wedge a_1 \neq a_2$$

```
type bExp =
  | TT
  | Eq of (aExp * aExp)
  | Lt of (aExp * aExp)
  | Conj of (bExp * bExp)
  | Not of bExp
```

```
let (.=.) a b = Eq (a, b)
let (.<.) a b = Lt (a, b)
let (.<=.) a b =
  a .<. b .||. (a .==. b)
let (.<=.) a b = ~~(a .<. b)
let (.>.) a b =
  (a .<>. b) .&&. (a .>=. b)
```

Statement implementation

$S ::= \epsilon$

| declare v

| $v \leftarrow a$

| $S; S$

| if b then S else S

| while b do S

`type stmtnt =`

| Skip

| Declare `of` string

| Assign `of` string * aExpr

| Seq `of` stmtnt * stmtnt

| If `of` bExp * stmtnt * stmtnt

| While `of` bExp * stmtnt

Statement implementation

 $S ::= \epsilon$
 $\mid \text{declare } v$
 $\mid v \leftarrow a$
 $\mid S; S$
 $\mid \text{if } b \text{ then } S \text{ else } S$
 $\mid \text{while } b \text{ do } S$
`type stmtnt =`
`| Skip`
`| Declare of string`
`| Assign of string * aExpr`
`| Seq of stmtnt * stmtnt`
`| If of bExp * stmtnt * stmtnt`
`| While of bExp * stmtnt`
 $\text{if } b \text{ then } S = \text{if } b \text{ then } S \text{ else } \epsilon$
`let IT b s = If (b, s, Skip)`

State

In order to keep track of program state
we need a mapping from program
variables to values

State

In order to keep track of program state
we need a mapping from program
variables to values

```
type state = Map<string, int>
```

Arithmetic expressions

```
type aExp =  
  | N of int  
  | V of string  
  | Add of (aExp * aExp)  
  | Mul of (aExp * aExp)  
  | Div of (aExp * aExp)
```

Arithmetic expressions

Arithmetic expressions can be evaluated
in the context of a state

`evalA : aExp -> state -> int`

```
type aExp =  
| N of int  
| V of string  
| Add of (aExp * aExp)  
| Mul of (aExp * aExp)  
| Div of (aExp * aExp)
```

```
let rec evalA arith st =  
  match arith with  
  | N n -> n  
  | V v -> Map.find v st  
  | Add(a, b) ->  
    let va = evalA a st  
    let vb = evalA b st  
    va + vb
```

...

Arithmetic expressions

Arithmetic expressions can be evaluated
in the context of a state

`evalA : aExp -> state -> int`

```
let rec evalA arith st =
  match arith with
  | N n -> n
  | V v -> Map.find v st
  | Add(a, b) ->
      let va = evalA a st
      let vb = evalA b st
      va + vb
```

...

```
let rec evalA arith =
  match arith with
  | N n -> fun st -> n
  | V v -> fun st ->
      Map.find v st
  | Add(a, b) ->
      fun st ->
        let va = evalA a st
        let vb = evalA b st
        va + vb
```

Arithmetic expressions

Arithmetic expressions can be evaluated
in the context of a state

```
evalA : aExp -> state -> int
let rec evalA arith =
  match arith with
  | N n -> fun st -> n
  | V v -> fun st ->
      Map.find v st
  | Add(a, b) ->
      fun st ->
        let va = evalA a st
        let vb = evalA b st
        va + vb

let rec evalA arith =
  match arith with
  | N n -> fun _ -> n
  | V v -> Map.find v
  | Add(a, b) ->
      fun st ->
        (evalA a st) +
        (evalA b st)
  ...
```

Arithmetic expressions

Arithmetic expressions can be evaluated
in the context of a state

```

evalA : aExp -> state -> int
let rec evalA arith =
  match arith with
  | N n -> fun _ -> n
  | V v -> Map.find v
  | Add(a, b) ->
      fun st ->
        (evalA a st) +
        (evalA b st)
  ...

```

```

let rec evalA arith =
  match arith with
  | N n -> fun _ -> n
  | V v -> Map.find v
  | Add(a, b) ->
      fun st ->
        (+)
        (evalA a st)
        (evalA b st)
  ...

```

Arithmetic expressions

Arithmetic expressions can be evaluated
in the context of a state

```
evalA : aExp -> state -> int

let rec evalA arith =
  match arith with
  | N n -> fun _ -> n
  | V v -> Map.find v
  | Add(a, b) ->
      fun st ->
        (+)
        (evalA a st)
        (evalA b st)
  ...
```

```
let binop : ('a -> 'b -> 'c)
           -> (state -> 'a)
           -> (state -> 'b)
           -> state -> 'c =
  fun f x y s -> f (x s) (y s)
```

Arithmetic expressions

```
let rec evalA arith =
  match arith with
  | N n -> fun _ -> n
  | V v -> Map.find v
  | Add(a, b) ->
      fun st ->
        (+)
        (evalA a st)
        (evalA b st)
  ...
```

```
let binop : ('a -> 'b -> 'c)
           -> (state -> 'a)
           -> (state -> 'b)
           -> state -> 'c =
  fun f x y s -> f (x s) (y s)

let rec evalA arith =
  match arith with
  | N n -> fun _ -> n
  | V v -> Map.find v
  | Add(a, b) -> binop (+)
                    (evalA a)
                    (evalA b)
```

Arithmetic expressions

Arithmetic expressions can be evaluated
in the context of a state

`evalA : aExp -> state -> int`

`let binop f x y s = f (x s) (y s)`

`let rec evalA =`

`function`

`| N n -> fun _ -> n`

`| V v -> Map.find v`

`| Add(a, b) -> binop (+) (evalA a) (evalA b)`

`| Sub(a, b) -> binop (-) (evalA a) (evalA b)`

`| Mul(a, b) -> binop (*) (evalA a) (evalA b)`

`| Div(a, b) -> binop (/) (evalA a) (evalA b)`

Evaluation functions

```
arithEval  : aExp -> state -> int option  
boolEval   : bExp -> state -> bool option  
stmntEval  : stmnt -> state -> state option
```

```
let rec arithEval =  
    <Assignment for this week>  
let rec boolEval =  
    <Assignment for this week>  
let rec stmntEval =  
    <Assignment for this week>
```

Questions?